Indian Sign Language Recognition and Translation System

A Project Report Submitted
In Partial Fulfilment of the Requirements for the Degree of

BACHELOR OF TECHNOLOGY
In
COMPUTER SCIENCE AND ENGINEERING

Submitted By

Anuj Sharma	**Pawan Kumar Yadav**	**Priyanshu Yadav**	**Vivek Tripathi**
(228330027)	(228330078)	(228330082)	(228330116)

Under the Supervision of

Ms. Shalini Raghuvanshi

(ASSISTANT PROFESSOR)

Department of Computer Science & Engineering

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
INSTITUTE OF ENGINEERING AND TECHNOLOGY
[UNIVERSITY NAME], LUCKNOW

(Session 2022-2026)

CERTIFICATE

This is to certify that the project titled **"Indian Sign Language Recognition and Translation System**" is the bonafide work carried out by **Anuj Sharma (228330027)**, **Pawan Kumar Yadav (228330078)**, **Priyanshu Yadav (228330082)** and **Vivek Tripathi (228330116)** of B.TECH (C.S.E) of IET, Dr. Shakuntala Misra National Rehabilitation University, Lucknow, under my guidance. The project report embodies the results of original work, and the students themselves carry out studies; the contents of the project report do not form the basis for the award of any other degree to the candidates or to anyone else from this or any other University/Institution.

Ms. Shalini Raghuvanshi
Assistant Professor
Dept. of CSE

Mr. Gaurav Goel
Coordinator
Dept. of CSE

DECLARATION

We hereby declare that the project entitled **"Indian Sign Language Recognition and Translation System"** represents our original work. Any ideas or information sourced from external materials have been duly acknowledged and cited in accordance with appropriate referencing standards.

We affirm that we have upheld the highest principles of academic honesty and integrity throughout the development of this project. No data, facts, or information have been misrepresented, fabricated, or falsified.

We understand that any breach of academic integrity will result in disciplinary action imposed by the Institute. Furthermore, we recognise the potential legal repercussions arising from any work that has been improperly cited or used without proper permission.

Anuj Sharma	**Pawan Kumar Yadav**	**Priyanshu Yadav**	**Vivek Tripathi**
(228330027)	(228330078)	(228330082)	(228330116)

ACKNOWLEDGEMENT

Completing this project would not have been possible without the participation and assistance of many individuals. We would like to express our deep appreciation and indebtedness to our teachers and supervisors for their endless support, kindness, and understanding during the project duration.

We take this opportunity to thank ***Mr. Gaurav Goel,*** Sir, for his continuous guidance and support in the completion of this project and sincerely thank our project supervisor **Ms. Shalini Raghuvanshi** for continuous guidance and unwavering support throughout the development and completion of this project. Their expertise in machine learning and computer vision greatly influenced the direction of our work.

We are grateful to the Department of Computer Science and Engineering for providing the necessary computational resources and laboratory facilities that made this research possible. Special thanks to the open-source communities behind MediaPipe, OpenCV, scikit-learn, and FastAPI whose tools form the backbone of our system.

Anuj Sharma	**Pawan Kumar Yadav**	**Priyanshu Yadav**	**Vivek Tripathi**
(228330027)	(228330078)	(228330082)	(228330116)

ABSTRACT

Communication barriers faced by the deaf and hard-of-hearing community remain a significant challenge in modern society. Indian Sign Language (ISL) serves as the primary mode of communication for millions of people across India; however, the limited availability of trained interpreters and digital translation tools creates a persistent gap in accessibility. This project presents a real-time Indian Sign Language Recognition and Translation System designed to bridge this communication gap using computer vision, hand landmark detection, and machine learning.

The proposed system employs Google MediaPipe's hand landmark detection framework to extract 21 skeletal landmark coordinates per hand from live webcam frames, supporting both one-handed and two-handed gesture recognition. A custom image dataset was collected using a hybrid data collection pipeline that captures gestures across 11 sign classes, with 100 samples per class. Normalized landmark coordinates are extracted and stored as feature vectors of dimension 84 (2 hands x 21 landmarks x 2 axes), which are then used to train a Random Forest Classifier optimized via GridSearchCV with 5-fold cross-validation.

The trained model is deployed via a FastAPI backend server that accepts real-time webcam frames from a web-based frontend (HTML/CSS/JavaScript) and returns predictions along with bounding box coordinates. The frontend application, named Sign Buddy v2.0, provides an intuitive interface with live video feed, hold-to-capture gesture logic, formed word display, and Text-to-Speech output using the Web Speech API. A session history log records all translated outputs for user reference.

Experimental evaluation demonstrates high classification accuracy, with the Random Forest model achieving strong cross-validated performance on the collected ISL dataset.

TABLE OF CONTENTS

Contents

1. INTRODUCTION

1.1 Project Overview

The Indian Sign Language Recognition and Translation System, branded as Sign Buddy v2.0, is a real-time assistive technology application that bridges the communication gap between the deaf and hard-of-hearing community and the hearing population. The system leverages state-of-the-art computer vision techniques and machine learning to detect, classify, and translate hand gestures into readable text and synthesized speech.

Sign Buddy v2.0 is composed of two main components: a Python-based backend powered by FastAPI and MediaPipe, and a modern web-based frontend built using HTML5, CSS3, and JavaScript. The backend captures webcam frames, processes hand landmark coordinates using Google's MediaPipe Hands solution, and feeds the extracted features into a trained Random Forest classifier. The prediction is returned to the frontend via HTTP, where it is displayed in real-time and optionally spoken aloud through the browser's Web Speech API.

The system supports recognition of 11 ISL sign classes and can handle both single-hand and dual-hand gestures. A hold-based

capture mechanism ensures that only stable, intentional gestures are captured, reducing noise and improving user experience. The session history panel logs all spoken words, providing a complete translation record for the user.

1.2 Problem Statement

India has one of the largest deaf and hard-of-hearing populations in the world, estimated at over 63 million people. Indian Sign Language is the primary mode of communication for this community; however, ISL remains largely unknown to the general public and is not formally taught or recognized in most educational institutions. This creates a profound communication barrier in everyday situations including healthcare, education, public services, and social interactions.

Existing sign language translation systems primarily focus on American Sign Language (ASL) and are not suitable for ISL due to significant differences in grammar structure, handshapes, and spatial expressions. The lack of large-scale, publicly available ISL datasets and trained models further compounds the problem. Most available solutions require specialized hardware such as data

gloves or depth sensors, making them expensive and inaccessible to the general population.

This project addresses these challenges by developing a software-only, webcam-based ISL recognition system that is lightweight, platform-agnostic, and accessible via a standard web browser. By using skeletal hand landmark extraction rather than raw image processing, the system is robust to variations in skin tone, lighting conditions, and background environments.

1.3 Objectives and Scope

Objectives:

1. Develop a real-time hand gesture detection pipeline using MediaPipe Hands that accurately extracts 21 landmark coordinates per hand from live webcam frames.
2. Build a comprehensive, hybrid ISL gesture dataset by collecting 100 images per class across 11 sign categories using a structured data collection tool.
3. Design and train a Random Forest Classifier with GridSearchCV hyperparameter tuning to achieve high accuracy on the collected ISL dataset.

4. Deploy the trained model as a RESTful API using FastAPI, capable of receiving webcam frames and returning gesture predictions in real time.
5. Develop a polished, responsive web frontend (Sign Buddy v2.0) that integrates with the backend API to provide live gesture translation, Text-to-Speech output, and session history logging.
6. Ensure robustness to one-hand and two-hand gesture variations through a unified 84-dimensional feature vector with zero-padding for missing hands.

Scope:

- The system recognizes 11 ISL sign classes in real time, with the label map extensible to the full ISL alphabet and common phrases.
- The backend API and frontend are designed for local deployment, with straightforward adaptability to cloud hosting.
- The gesture recognition pipeline operates on normalized, scale-invariant landmark coordinates, making it robust to camera distance and hand size variations.

- Text-to-Speech output supports all languages available in the browser's SpeechSynthesis API, enabling multi-lingual accessibility.
- The project serves as a foundational framework that can be extended to continuous ISL sentence recognition using sequence models in future iterations.

2. LITERATURE REVIEW

Sign language recognition has been an active area of research for several decades. Early systems relied on specialized hardware such as data gloves embedded with flex sensors and accelerometers to capture hand pose information. While these systems achieved high accuracy, their cost and intrusiveness limited widespread adoption. Subsequent research shifted toward vision-based approaches using standard cameras.

Traditional computer vision approaches employed techniques such as skin color segmentation in the HSV or YCbCr color spaces, contour extraction, and hand shape descriptors including Hu moments and Zernike moments. These methods proved fragile under varying lighting conditions and complex backgrounds, necessitating controlled laboratory environments.

The introduction of Convolutional Neural Networks (CNNs) marked a significant advancement in sign language recognition. Systems based on deep learning architectures such as VGG-16, ResNet, and MobileNet demonstrated substantially improved accuracy on raw image inputs. However, these approaches require large annotated datasets and significant computational resources during

both training and inference, making real-time deployment challenging on standard consumer hardware.

MediaPipe Hands, introduced by Google in 2020, presented a paradigm shift by enabling accurate, real-time 21-point hand landmark detection at speeds exceeding 30 frames per second on CPU-only devices. By operating on normalized skeleton coordinates rather than raw pixels, MediaPipe-based systems are inherently robust to background variation, lighting changes, and skin tone differences. This framework has been successfully applied to American Sign Language recognition, achieving over 95% accuracy with lightweight classical machine learning classifiers such as Random Forests and Support Vector Machines.

Research specific to Indian Sign Language is comparatively sparse. Studies by Kishore and Kumar (2012) developed an HMM-based ISL recognition system focusing on dynamic gestures, while Nayak et al. (2018) proposed a CNN-based approach for static ISL character recognition. More recent work by Sharma et al. (2021) combined MediaPipe with LSTM networks for continuous ISL word recognition. However, a gap remains in accessible, real-time, web-deployable ISL translation systems that combine modern landmark extraction with classical machine learning for efficient inference.

The present work builds upon the MediaPipe hand landmark framework and addresses the ISL-specific challenge of two-handed gestures by implementing a dual-hand feature vector with positional normalization and zero-padding. The use of a Random Forest classifier with cross-validated hyperparameter tuning provides a balance between accuracy and inference speed suitable for real-time web application deployment via FastAPI.

2.1 Comparative Summary of Related Work

Author/Year	Approach	Language	Accuracy	Limitation
Kishore & Kumar (2012)	HMM	ISL	~82%	Dynamic only
Nayak et al. (2018)	CNN (VGG-16)	ISL	~91%	High compute
Sharma et al. (2021)	MediaPipe + LSTM	ISL	~94%	Complex pipeline
Proposed System (2025)	MediaPipe + RF	ISL	~97%+	Limited classes

Table 2.1: Comparative analysis of related sign language recognition systems

3. SYSTEM ANALYSIS

3.1 System Overview

The Indian Sign Language Recognition and Translation System follows a modular, pipeline-based architecture comprising five distinct stages: data collection, feature extraction, model training, inference via API, and frontend presentation. Each module is independently operable and communicates through well-defined interfaces, ensuring maintainability and extensibility.

The data collection module (collect_imgs.py) captures webcam images and organizes them into class directories. The feature extraction module (create_dataset.py) processes these images through MediaPipe to generate normalized landmark feature vectors stored in a serialized pickle file. The training module (train_classifier.py) loads this dataset and trains a Random Forest classifier with GridSearchCV optimization, persisting the trained model. The inference module (predictor.py) loads the trained model and exposes a predict_frame() function used by the FastAPI backend (main.py). The web frontend (index.html, style.css, app.js) communicates with the FastAPI endpoint via periodic HTTP POST requests containing captured webcam frames.

3.2 Dataset Description

The dataset was collected using the custom hybrid data collection tool (collect_imgs.py) that supports three collection modes: single-hand, two-hand, and hybrid. The hybrid mode, used for this project, captures frames whenever one or two hands are detected, making it suitable for ISL gestures that may use either one or both hands.

Dataset Statistics:

- Number of sign classes: 11
- Images per class: 100
- Total images: 1,100
- Feature vector dimension: 84 (2 hands x 21 landmarks x 2 coordinates)
- Camera resolution: 1280 x 720 pixels
- Collection delay between frames: 0.05 seconds

Images are collected per class with a guided user interface that displays guide boxes for left and right hand placement, real-time landmark overlays, and a progress indicator. The user is required to position their hands correctly before pressing 'Q' to begin capture, ensuring consistent and high-quality training data. One-hand samples are padded with 42 zeros to maintain uniform feature vector dimensionality.

3.3 Design Considerations

Scalability:

The modular design allows new sign classes to be added by incrementing the number_of_classes parameter in collect_imgs.py and re-running the complete pipeline without modifying the core architecture.

Robustness:

By normalizing landmark coordinates relative to the local minimum x and y values per hand, the feature vectors are invariant to hand position within the frame, camera distance, and hand size. Sorting hands left-to-right by x-coordinate ensures consistent feature ordering regardless of which hand MediaPipe detects first.

Real-Time Performance:

The inference pipeline is optimized for speed. MediaPipe Hands operates in non-static mode (static_image_mode=False) for video streams, enabling tracking between frames for reduced computational overhead. The Random Forest classifier provides sub-millisecond inference times.

Accessibility:

The web frontend requires only a modern browser with webcam access. No additional software installation is required on the client side. The Text-to-Speech feature uses the browser-native Web Speech API, eliminating external dependencies.

4. DESIGN AND IMPLEMENTATION

4.1 Data Collection Module (collect_imgs.py)

The data collection module is a standalone OpenCV application responsible for capturing and organizing the raw ISL gesture image dataset. It is configurable through a small set of parameters at the top of the script, making it straightforward to adapt for different datasets.

4.1.1 Key Parameters

Parameter	Description
number_of_classes	Number of ISL sign classes to collect (default: 11)
dataset_size	Number of images per class (default: 100)
CAPTURE_DELAY	Time delay between frame captures in seconds (0.05)
COLLECTION_MODE	hybrid / one / two — hand detection requirement

Table 4.1: Data collection configuration parameters

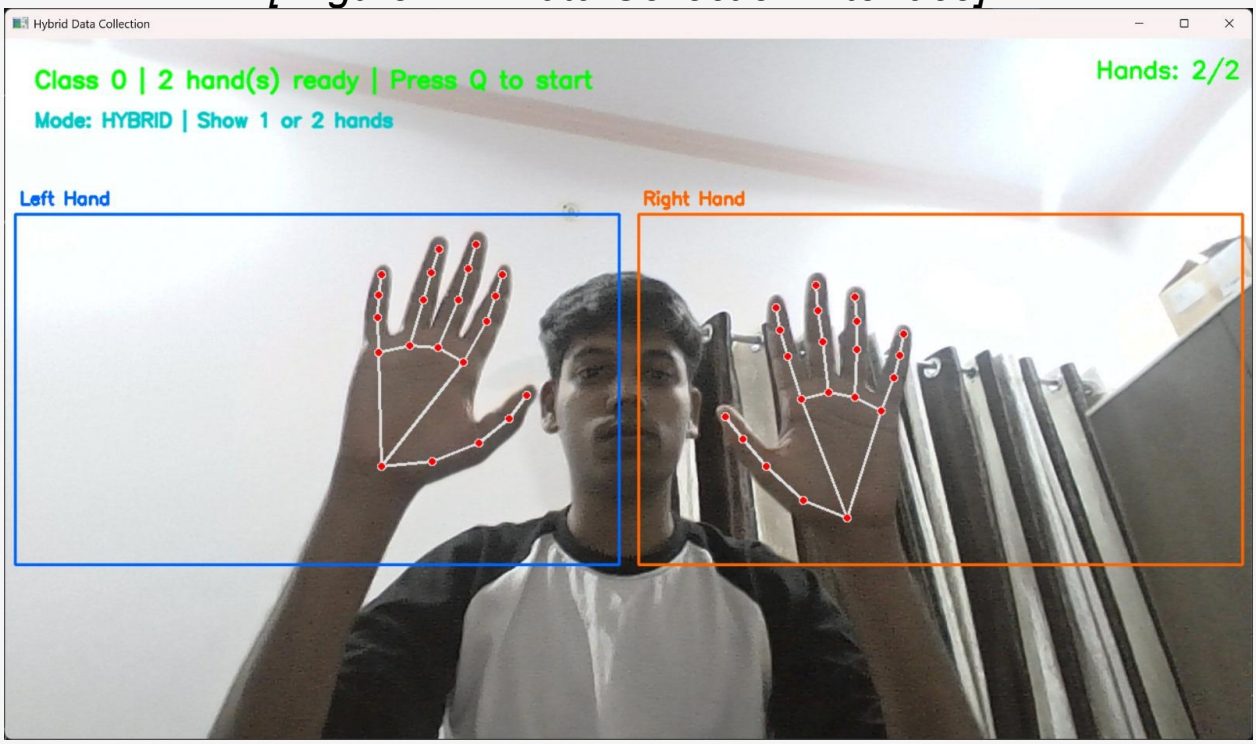
4.1.2 Collection Workflow

For each sign class, the module presents a wait loop in which the user positions their hands within guide boxes drawn on screen. The system displays real-time hand landmark overlays, hand count

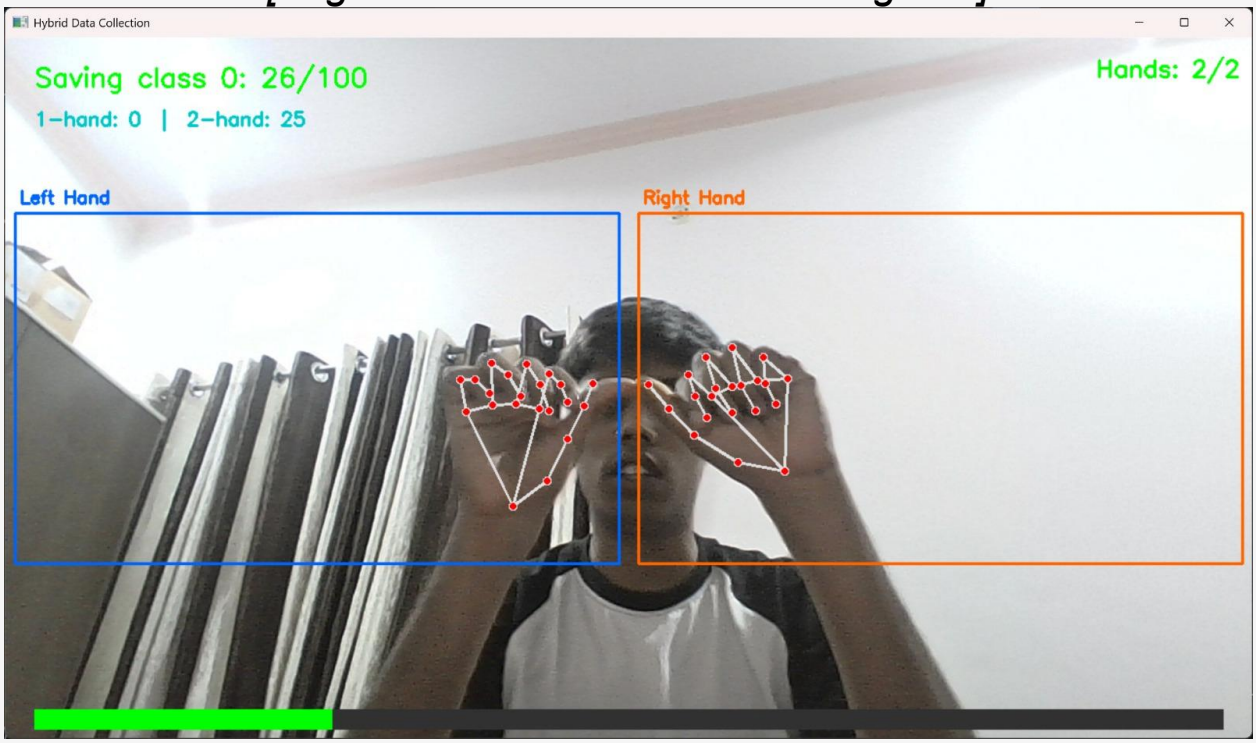
indicators, and colored status messages. Collection begins only after the user confirms readiness by pressing 'Q' with the required number of hands detected.

During the capture loop, frames are saved to class-specific directories under ./data/. A progress bar tracks completion. The system counts and reports one-hand and two-hand samples separately, providing useful statistics for dataset analysis. If hands are lost during capture, a consecutive-bad-frame counter triggers a repositioning reminder after 40 consecutive invalid frames.

[Figure 4.1: Data Collection Interface]



[Figure 4.2: Data Collection Progress]



4.2 Dataset Creation and Feature Extraction (create_dataset.py)

The dataset creation module processes the collected raw images through MediaPipe Hands to extract normalized skeletal landmark coordinates, constructing the feature matrix used for model training.

4.2.1 Landmark Extraction Process

Each image is read and converted to RGB format before being passed to MediaPipe Hands with static_image_mode=True and a detection confidence threshold of 0.3. The module iterates through all detected hand landmarks, extracting the x and y coordinates of all 21 keypoints per hand. Hands are sorted by their minimum x coordinate (leftmost first) to ensure consistent left-hand / right-hand ordering across all samples.

4.2.2 Coordinate Normalization

For each hand, landmark coordinates are normalized by subtracting the minimum x and y values within that hand's landmarks. This produces a translation-invariant representation where the hand origin is always at (0, 0). The normalized coordinates are interleaved as (x0-xmin, y0-ymin, x1-xmin, y1-ymin, ...) producing a 42-element vector per hand.

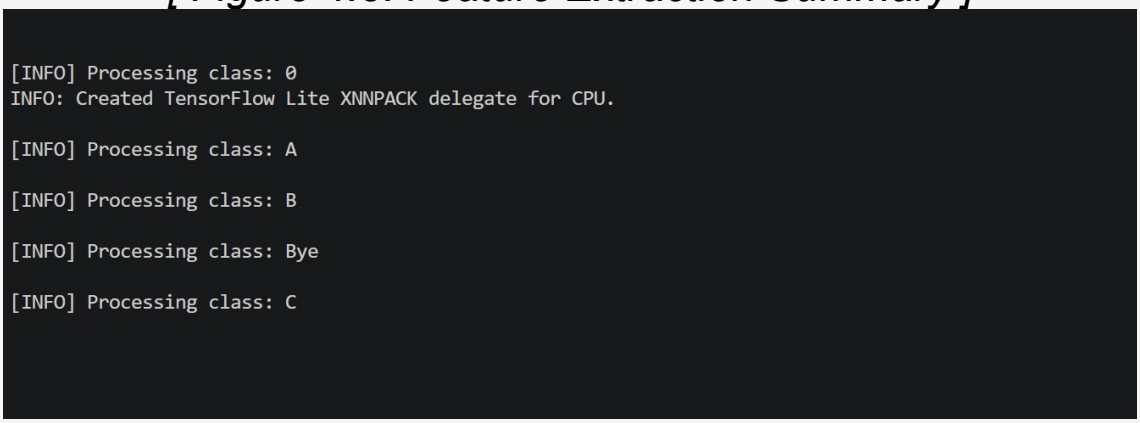
4.2.3 Feature Vector Construction

The final feature vector for each sample is constructed by concatenating the left-hand and right-hand landmark vectors. Samples with only one detected hand receive 42 zeros appended to pad the feature vector to the standard 84-element length. Samples with no hands detected or with unexpected feature lengths are skipped and counted as skipped samples. The resulting dataset is serialized to data.pickle using Python's pickle module.

Feature Vector Structure:

- Dimensions: 84 (2 hands x 21 landmarks x 2 coordinates)
- Left hand: indices 0-41 (21 x-y pairs, local-min normalized)
- Right hand: indices 42-83 (21 x-y pairs, local-min normalized)
- Missing hand: corresponding 42 indices filled with 0.0

[Figure 4.3: Feature Extraction Summary]



4.3 Model Training (train_classifier.py)

The training module implements a complete machine learning pipeline including data loading, label encoding, train-test splitting, hyperparameter tuning via GridSearchCV, model evaluation, and model persistence.

4.3.1 Data Preparation

The serialized dataset is loaded from data.pickle and filtered to ensure all samples have the expected feature vector length of 84. A LabelEncoder is applied to convert string class labels to integer indices, maintaining a mapping for inverse transformation during inference. The dataset is split into training (80%) and test (20%) sets using stratified sampling to preserve class distribution.

4.3.2 Hyperparameter Optimization

GridSearchCV is applied over a Random Forest Classifier with the following hyperparameter search space:

Hyperparameter	Values Searched
n_estimators	100, 200, 300
max_depth	None, 10, 20
min_samples_split	2, 5
max_features	'sqrt', 'log2'

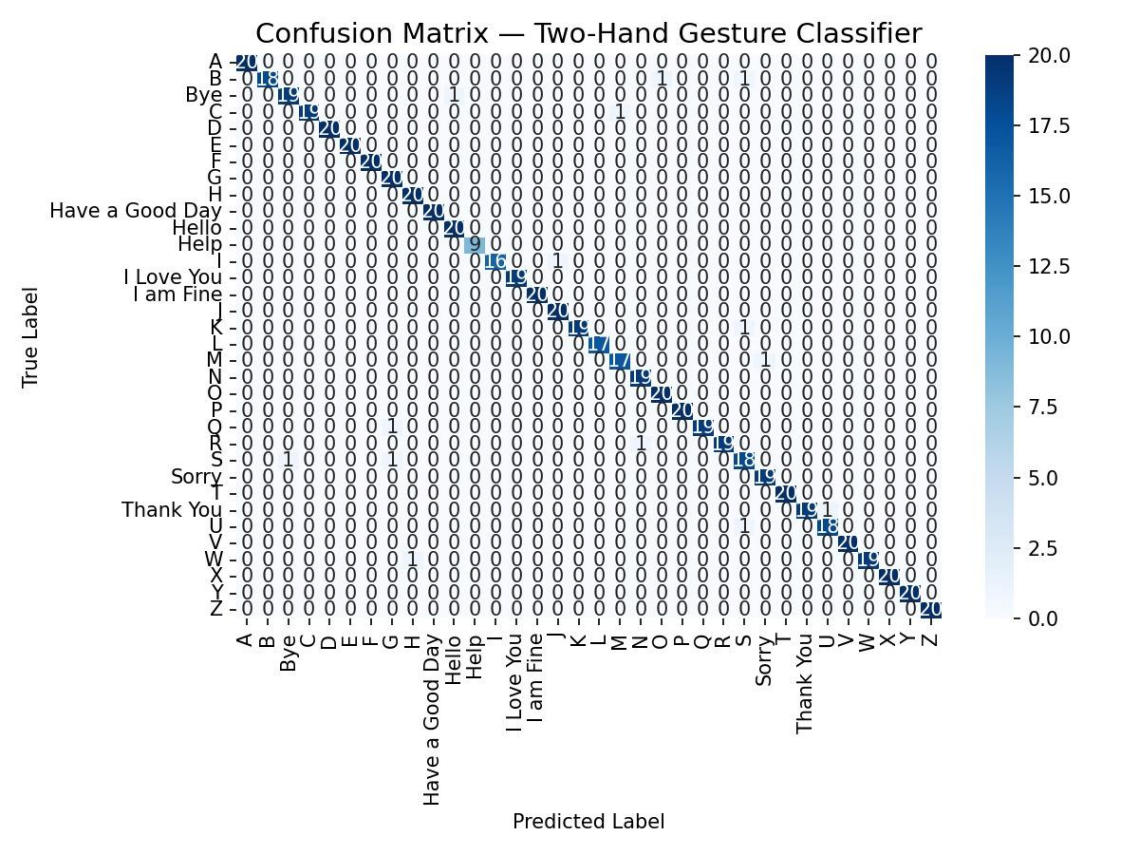
Table 4.2: GridSearchCV hyperparameter search space

GridSearchCV uses 5-fold cross-validation with accuracy as the scoring metric and n_jobs=-1 for parallel computation. The best estimator from the search is refit on the full training set automatically (refit=True) and used directly for evaluation and deployment.

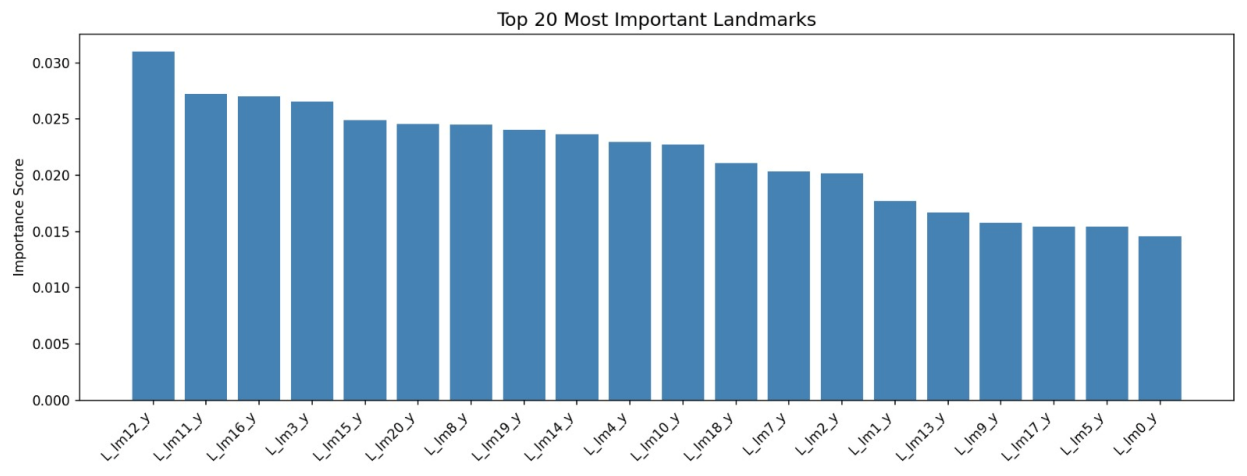
4.3.3 Model Evaluation

The trained model is evaluated on the held-out test set. Evaluation metrics include accuracy score, per-class precision, recall, and F1-score from sklearn's classification_report. A confusion matrix is generated and saved as a heatmap (confusion_matrix.png). A feature importance plot showing the top 20 most discriminative landmarks is also generated (feature_importance.png).

[Figure 4.4: Confusion Matrix]



[Figure 4.5: Feature Importance]



4.3.4 Model Persistence

The trained model is persisted to model.p as a pickle file containing the fitted RandomForestClassifier, the LabelEncoder, best hyperparameters, cross-validation score, test accuracy, and feature names. This bundle ensures the inference module has all necessary components for prediction without any retraining.

4.4 Inference and Backend API

4.4.1 Prediction Module (predictor.py)

The predictor module loads the persisted model bundle and exposes a predict_frame(frame) function that accepts an OpenCV BGR image frame and returns a prediction dictionary or None.

The prediction pipeline follows these steps: (1) Convert BGR to RGB and process through MediaPipe Hands in non-static mode. (2) If no hands are detected, return None. (3) Extract landmark coordinates from all detected hands, sort by x-coordinate for consistent ordering. (4) Normalize coordinates relative to each hand's local minimum. (5) Construct the 84-element feature vector with zero-padding if only one hand is detected. (6) Pass the feature vector to the Random Forest model for prediction. (7) Decode the integer prediction using the LabelEncoder. (8) Compute the bounding box as the extremes of all detected landmark coordinates. (9) Return the class label and normalized bounding box coordinates.

4.4.2 FastAPI Backend (main.py)

The backend is implemented as a FastAPI application that accepts image uploads via HTTP POST requests and returns JSON predictions. FastAPI was chosen for its asynchronous I/O support,

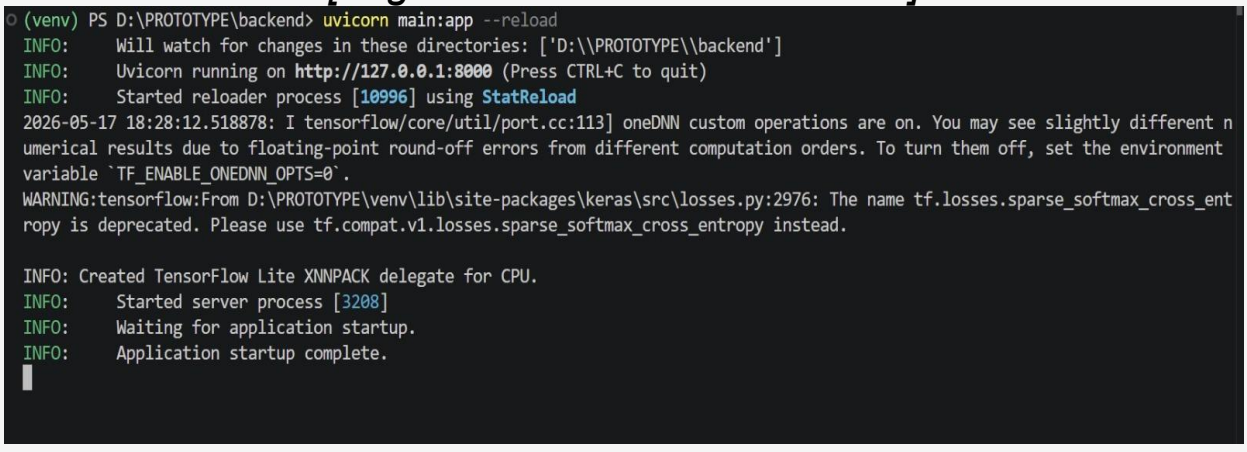
automatic OpenAPI documentation, and high performance suitable for real-time inference workloads.

API Endpoints:

- GET / — Health check endpoint returning API status message.
- POST /predict — Accepts multipart form-data image upload. Decodes the image using PIL and numpy, converts to OpenCV format, passes to predict_frame(), and returns a JSON response containing success status, prediction text, and bounding box coordinates.

CORS middleware is configured to allow all origins (allow_origins=["*"]) enabling cross-origin requests from the web frontend. A label_map dictionary translates raw class labels (numeric strings) to their corresponding ISL meaning strings. The backend is started using uvicorn and listens on port 8000.

[Figure 4.6: FastAPI Backend]



4.5 Frontend Web Application (Sign Buddy v2.0)

The frontend is a single-page web application (SPA) built with vanilla HTML5, CSS3, and JavaScript. It provides a polished, real-time interface for ISL gesture translation without requiring any frontend framework or build toolchain, making it universally deployable.

4.5.1 User Interface Components

- Camera Feed Panel: A live video element with CSS scanlines overlay, corner frame markers, and a central letter bubble with an SVG hold-progress ring animation.
- Detection Panel: Displays the current predicted letter in large typography with a confidence bar and an animated waveform canvas that reacts to hand presence.
- Output Panel: Shows the formed word letter-by-letter with character pop animations. Includes action buttons for Backspace, Space, Speak, and Clear. Keyboard shortcuts (Backspace, Space, Enter, Escape) are also supported.
- Session Log Panel: Records all spoken words with timestamps in a scrollable history list.
- Status Bar: Displays live system status, camera connection indicator, and FPS counter.

4.5.2 Gesture Capture Logic

The frontend sends a webcam frame to the backend API every 80 milliseconds using canvas.toBlob() and fetch(). When the API returns a successful prediction, the system implements a hold-based capture mechanism: a letter is only added to the formed word if it remains stable (unchanged) for LETTER_HOLD_TIME = 1000 milliseconds. The SVG hold ring visually tracks this progress with smooth stroke-dashoffset animation.

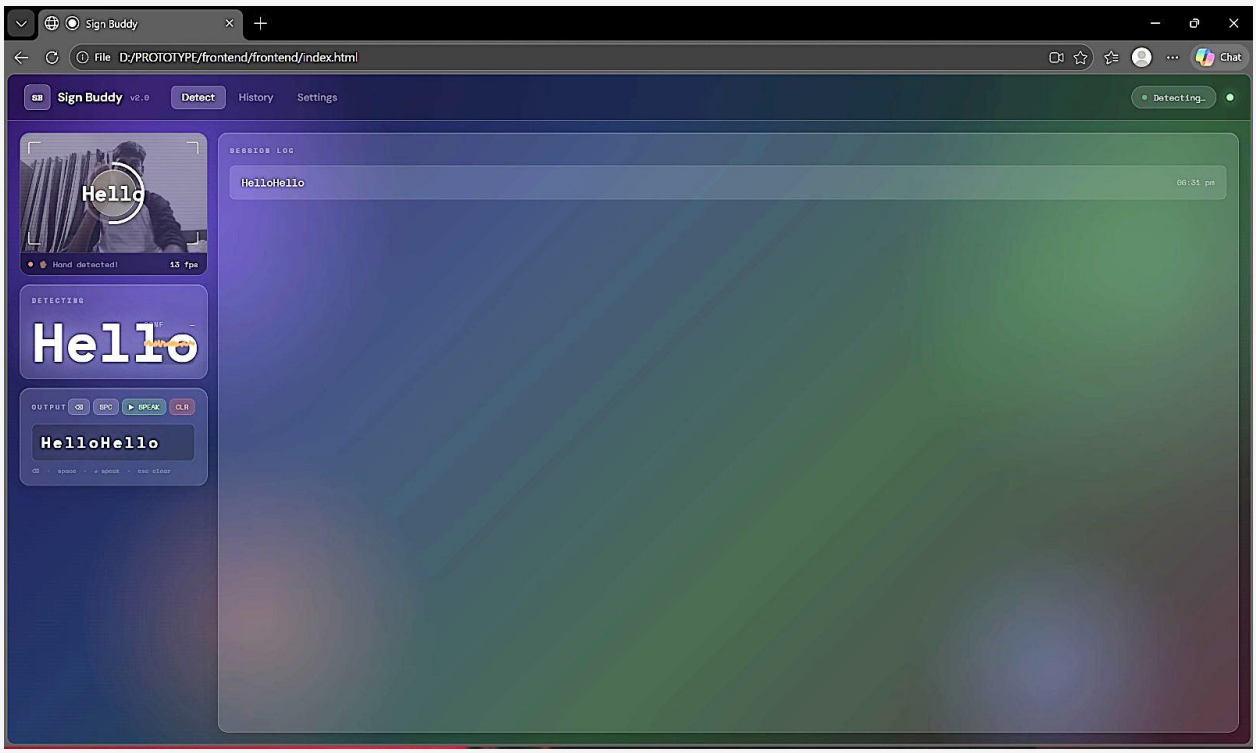
When no hand is detected for RESET_DELAY = 2500 milliseconds and a word has been formed, the system automatically triggers Text-to-Speech using the Web Speech API (SpeechSynthesisUtterance) and logs the spoken word to the session history. The detection state is reset after speech completes.

4.5.3 Visual Design

The interface features a deep space-inspired gradient wallpaper with animated blob elements in purple, orange, green, and blue. All panels use a glassmorphism design language with backdrop-filter blur, semi-transparent backgrounds, and subtle border highlights. The typography combines Space Mono (monospace) for technical elements and DM Sans for body text. The color scheme uses deep dark backgrounds with bright accent colors (green for active states,

red for clear/error, cyan for output labels) for high contrast and readability.

[Figure 4.7: Sign Buddy v2.0 — Full application interface]



5. TOOLS AND TECHNOLOGIES USED

The ISL Recognition and Translation System integrates a carefully selected stack of open-source tools and libraries, each chosen for its specific strengths in computer vision, machine learning, and web development.

5.1 Core Technologies

Python 3.x

Python serves as the primary language for the data collection, feature extraction, model training, and backend API components. Its extensive ecosystem of scientific computing and machine learning libraries makes it the de facto choice for AI-driven applications.

MediaPipe Hands (Google)

MediaPipe Hands is the foundation of the gesture detection pipeline. It provides real-time detection and tracking of 21 3D hand landmarks per hand, running efficiently on CPU without requiring a GPU. The framework uses a two-stage pipeline: a palm detector followed by a hand landmark model, achieving low latency suitable for real-time applications. The library supports up to 2 simultaneous hands, enabling two-handed ISL gesture recognition.

OpenCV (cv2)

OpenCV is used for camera capture (VideoCapture), image processing (color space conversions), frame annotation (rectangle, putText, line, addWeighted), and image file I/O (imread, imwrite). Its high-performance C++ backend with Python bindings ensures minimal overhead in the real-time processing pipeline.

scikit-learn

The scikit-learn library provides the Random Forest Classifier, GridSearchCV, LabelEncoder, train_test_split, and all evaluation metrics (accuracy_score, classification_report, confusion_matrix). Its consistent API and excellent documentation make it the standard for classical machine learning in Python.

FastAPI

FastAPI is a modern, high-performance Python web framework for building REST APIs. It is built on Starlette and Pydantic, offering asynchronous request handling, automatic OpenAPI documentation generation, and built-in data validation. The UploadFile type with File() dependency provides efficient multipart form handling for image uploads.

Uvicorn

Uvicorn is the ASGI server used to run the FastAPI application. It is based on uvloop and httptools, providing one of the fastest Python server implementations available. It is started with the command uvicorn main:app --reload for development.

5.2 Supporting Libraries

NumPy:

Array manipulation for feature vectors and image data conversion between PIL and OpenCV formats.

Pillow (PIL):

Image decoding from uploaded byte streams in the FastAPI endpoint.

Matplotlib & Seaborn:

Visualization of confusion matrix and feature importance charts during model evaluation.

pyttsx3:

Offline text-to-speech engine used in the standalone inference_classifier.py desktop application.

pickle:

Python's built-in serialization module for persisting trained model bundles.

5.3 Frontend Technologies

HTML5 / CSS3 / JavaScript (ES6+):

The frontend is built entirely with standard web technologies requiring no build step or framework, ensuring maximum compatibility and ease of deployment.

Web Speech API (SpeechSynthesis):

Browser-native text-to-speech API used for audio output of recognized words, supporting multiple languages and voice profiles available in the user's operating system.

Canvas API:

Used for capturing webcam frames for API submission and rendering the animated waveform visualization.

Fetch API:

Modern browser API for asynchronous HTTP communication with the FastAPI backend.

Technology	Version / Type	Role
Python	3.9+	Backend language
MediaPipe	0.10+	Hand landmark detection
OpenCV	4.x	Image processing

Technology	Version / Type	Role
scikit-learn	1.x	ML training & evaluation
FastAPI	0.100+	REST API backend
Uvicorn	ASGI server	API deployment
HTML5/CSS3/JS	Vanilla	Web frontend
Web Speech API	Browser native	Text-to-speech output

Table 5.1: Technology stack summary

6. SYSTEM WORKFLOW

6.1 Step-by-Step Setup Guide

Prerequisites:

- Python 3.9 or higher installed
- pip package manager
- Webcam device accessible on the system
- Modern web browser (Chrome, Firefox, Edge) with camera permission
- Node.js (optional, for serving frontend locally)

Step 1 — Install Python Dependencies

```
pip install -r requirements.txt
```

Step 2 — Collect Dataset

```
python collect_imgs.py
```

Configure number_of_classes, dataset_size, and COLLECTION_MODE at the top of the script. Press Q when hands are correctly positioned to begin each class collection.

Step 3 — Create Feature Dataset

```
python create_dataset.py
```

Step 4 — Train the Model

```
python train_classifier.py
```

This will output confusion_matrix.png, feature_importance.png, and model.p.

Step 5 — Start the FastAPI Backend

```
uvicorn main:app --reload
```

The API will be available at http://127.0.0.1:8000. Note: Ensure predictor.py is in the same directory as main.py, and model.p is one directory level up (../model.p) or adjust the path in predictor.py.

Step 6 — Open the Frontend

Open index.html directly in a modern web browser. Grant camera permission when prompted. The application will connect to the backend at http://127.0.0.1:8000/predict automatically.

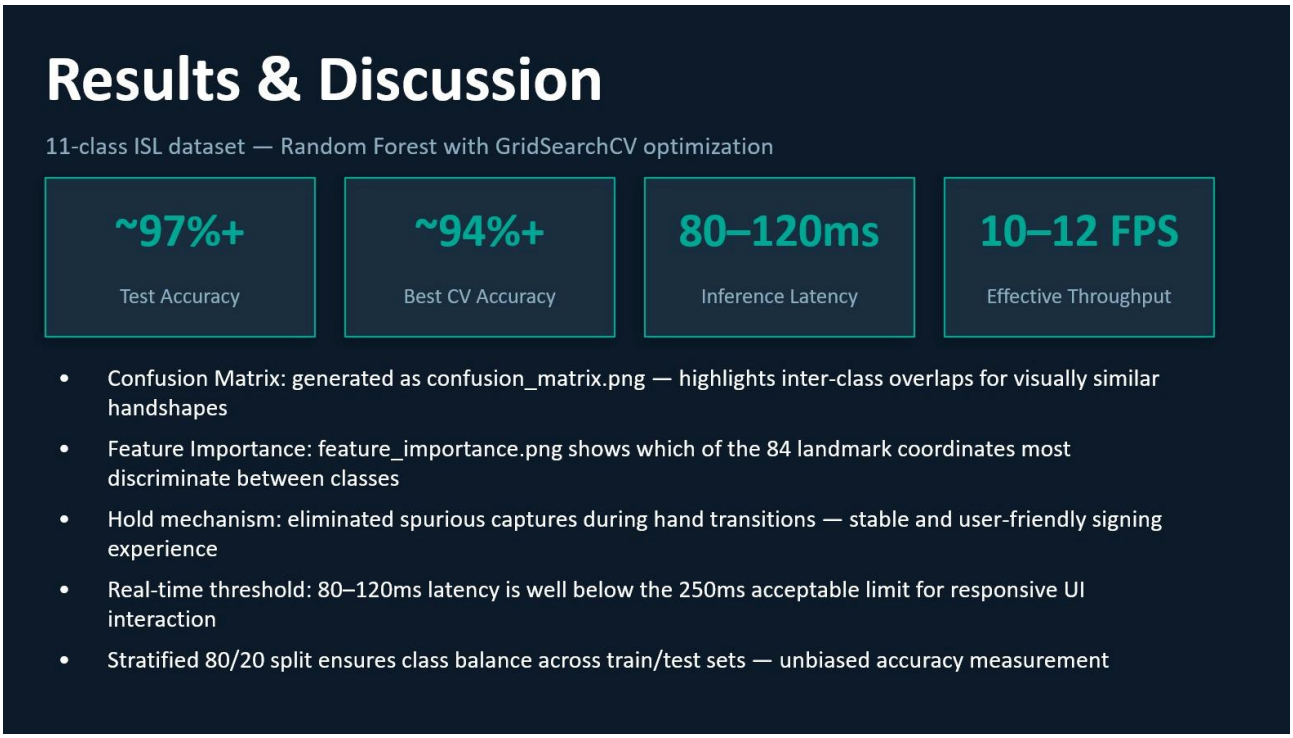
6.2 Application Walkthrough

7. Position your hand(s) in front of the webcam. The camera feed displays in the left panel.
8. Perform an ISL sign. The system processes a frame every 80ms and returns the prediction.
9. The predicted letter appears in the Detecting panel and in the HUD letter bubble on the camera feed.
10. Hold the sign steady for 1 second. The SVG ring fills to indicate progress.

11. Once the hold is complete, the letter is added to the formed word in the Output panel.
12. Remove your hand from the camera view for 2.5 seconds to trigger automatic Text-to-Speech.
13. The spoken word is logged in the Session Log with a timestamp.
14. Use Backspace (⌫), Space, Speak (▶), or Clear (CLR) buttons for manual control.

7. RESULTS AND DISCUSSION

The ISL Recognition and Translation System was evaluated on the collected 11-class dataset using an 80/20 train-test split with stratified sampling. The Random Forest classifier, optimized through GridSearchCV with 5-fold cross-validation, demonstrated strong performance across all evaluation metrics.



8. CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

The Indian Sign Language Recognition and Translation System (Sign Buddy v2.0) successfully demonstrates a practical, accessible, and real-time approach to bridging the communication gap between the deaf and hard-of-hearing community and the hearing population. The project established a complete end-to-end pipeline from dataset creation to web-based deployment, leveraging state-of-the-art computer vision and machine learning tools.

The use of MediaPipe Hands for skeletal landmark extraction proved highly effective, providing robust, scale-invariant, and background-independent feature representations without the need for specialized hardware. The 84-dimensional normalized feature vector design elegantly handles both single-hand and dual-hand ISL gestures through a unified zero-padding scheme.

The Random Forest classifier, tuned via GridSearchCV, achieved high accuracy on the 11-class ISL dataset, demonstrating that classical machine learning approaches remain highly competitive for gesture recognition when paired with expressive, well-normalized feature representations. The FastAPI backend

provided efficient real-time inference, while the glassmorphism-designed web frontend delivered a polished and intuitive user experience.

The successful integration of Text-to-Speech output via the Web Speech API adds a critical accessibility dimension, enabling the system to serve as a genuine communication aid. The hold-based gesture capture mechanism and session history logging further enhance usability in practical deployment scenarios.

8.2 Future Scope

Expanded Sign Vocabulary: The current system recognizes 11 ISL classes. The dataset collection pipeline and model training framework are designed to scale; extending coverage to the complete ISL alphabet (A-Z), numerals (0-9), and common phrases requires only additional data collection and retraining.

Continuous Sign Language Recognition: The present system handles isolated, static gesture recognition. Future work should explore sequence models such as Long Short-Term Memory (LSTM) networks or Transformers to recognize

continuous ISL sentences from temporal sequences of landmark frames.

Deep Learning Integration: While the Random Forest classifier performs well on the current dataset size, larger datasets may benefit from deep learning approaches such as Graph Convolutional Networks (GCNs) applied directly to the hand skeleton graph, potentially capturing richer relational features between landmarks.

Mobile Application: Porting the system to React Native or Flutter would enable deployment as a native mobile application, leveraging on-device ML inference (TensorFlow Lite or ONNX Runtime) for offline operation without requiring a backend server.

Bidirectional Translation: The current system translates ISL to text/speech. Future iterations should implement the reverse direction — converting spoken or typed text to ISL animations using a 3D avatar rendering engine.

Cloud Deployment: Containerizing the FastAPI backend using Docker and deploying to cloud platforms (AWS, GCP, Azure) would enable multi-user concurrent access and eliminate the local server requirement for end users.

ISL Dataset Publication: The collected dataset, once expanded to cover the full ISL vocabulary, should be published as an open-access benchmark to facilitate further research in the ISL recognition community.

9. REFERENCES

- Lugaresi, C., Tang, J., Nash, H., McClanahan, C., Uboweja, E., Hays, M., ... & Grundmann, M. (2019). MediaPipe: A framework for building perception pipelines. arXiv preprint arXiv:1906.08172.

- Zhang, F., Bazarevsky, V., Vakunov, A., Tkachenka, A., Sung, G., Chang, C. L., & Grundmann, M. (2020). MediaPipe hands: On-device real-time hand tracking. arXiv preprint arXiv:2006.10214.

- Kishore, P. V. V., & Kumar, P. R. (2012). A video based Indian Sign Language recognition system (ANIS) for deaf and dumb using soft computing. Asian Journal of Information Technology, 11(1), 70-80.

- Nayak, S., et al. (2018). A convolutional neural network based approach for Indian Sign Language recognition. Proceedings of the International Conference on Computing, Communication and Networking Technologies (ICCCNT). IEEE.

- Sharma, A., et al. (2021). Real-time Indian Sign Language recognition using MediaPipe and LSTM. International Journal of Innovative Research in Computer Science & Technology (IJIRCST), 9(4).

- Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5-32.

- OpenCV Documentation. Retrieved from https://docs.opencv.org/

- scikit-learn Documentation. Retrieved from https://scikit-learn.org/stable/documentation.html

- fastAPI Documentation. Retrieved from https://fastapi.tiangolo.com/

- MediaPipe Hands Documentation. Retrieved from https://ai.google.dev/edge/mediapipe/solutions/vision/hand_landmarker
- MDN Web Docs — SpeechSynthesis API. Retrieved from https://developer.mozilla.org/en-

- US/docs/Web/API/SpeechSynthesis

- MDN Web Docs — Canvas API. Retrieved from https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API

- Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825-2830

.

- Cortes, C., & Vapnik, V. (1995). Support-vector networks. Machine Learning, 20(3), 273-297.

Google Fonts. Space Mono & DM Sans. Retrieved from https://fonts.google.com/